

Title: HCI & CG Lab 01 Assignment

Student Name: **Abbas Khan**

Student-id: **CSE/89/2K24**

Submission-Date: **11-Sept-2026**

Task 1: Display PPI/DPI Calculator

Objective: To calculate total pixels, aspect ratio, display PPI/DPI, and display-density category.

- Code

```
import math

def get_positive_int(prompt):
    while True:
        try:
            value = int(input(prompt))
            if value > 0:
                return value
            print("Please enter a positive whole number.")
        except ValueError:
            print("Invalid input. Please enter a whole number.")

def get_positive_float(prompt):
    while True:
        try:
            value = float(input(prompt))
            if value > 0:
                return value
            print("Please enter a value greater than zero.")
        except ValueError:
            print("Invalid input. Please enter a numeric value.")

width_px = get_positive_int("Enter horizontal resolution (pixels): ")
height_px = get_positive_int("Enter vertical resolution (pixels): ")
```

```

diagonal_inches = get_positive_float("Enter physical diagonal size (inches): ")

total_pixels = width_px * height_px
gcd_value = math.gcd(width_px, height_px)
aspect_width = width_px // gcd_value
aspect_height = height_px // gcd_value

diagonal_pixels = math.sqrt(width_px ** 2 + height_px ** 2)
ppi = diagonal_pixels / diagonal_inches

if ppi < 100:
    category = "Low Density (Standard Monitor)"
elif ppi <= 200:
    category = "Medium Density (HD Display)"
else:
    category = "High Density (Retina / Mobile)"

print("\n--- DISPLAY METRICS ANALYSIS ---")
print(f"Total Pixel Count : {total_pixels:,} pixels")
print(f"Aspect Ratio      : {aspect_width}:{aspect_height}")
print(f"Calculated DPI     : {ppi:.2f} DPI")
print(f"Density Category   : {category}")

```

- Desktop output screenshot

```

PS C:\Users\ak539\OneDrive\Documents\HCI-CG-Lab-01-NumPy> C:\Users\ak539\AppData\Local\Programs\Python\Python313\python.exe c:/Users/ak539/OneDrive/Documents/HCI-CG-Lab-01-NumPy/task1_display_ppi.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio      : 16:9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)
PS C:\Users\ak539\OneDrive\Documents\HCI-CG-Lab-01-NumPy>

```

- Smartphone output screenshot

```

PS C:\Users\ak539\OneDrive\Documents\HCI-CG-Lab-01-NumPy> C:\Users\ak539\AppData\Local\Programs\Python\Python313\python.exe c:/Users/ak539/OneDrive/Documents/HCI-CG-Lab-01-NumPy/task1_display_ppi.py
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,962,440 pixels
Aspect Ratio      : 195:422
Calculated DPI    : 457.25 DPI
Density Category  : High Density (Retina / Mobile)
PS C:\Users\ak539\OneDrive\Documents\HCI-CG-Lab-01-NumPy>

```

Task 2: Synthetic Image Matrix Creation

Objective: To create a $300 \times 400 \times 3$ RGB image array using NumPy and fill its four quadrants with different colors.

- Code

```
import numpy as np

import matplotlib.pyplot as plt

height = 300
width = 400
channels = 3

image = np.zeros((height, width, channels), dtype=np.uint8)

mid_height = height // 2
mid_width = width // 2

# Top-left: Red
image[:mid_height, :mid_width] = [255, 0, 0]

# Top-right: Green
image[:mid_height, mid_width:] = [0, 255, 0]

# Bottom-left: Blue
image[mid_height:, :mid_width] = [0, 0, 255]

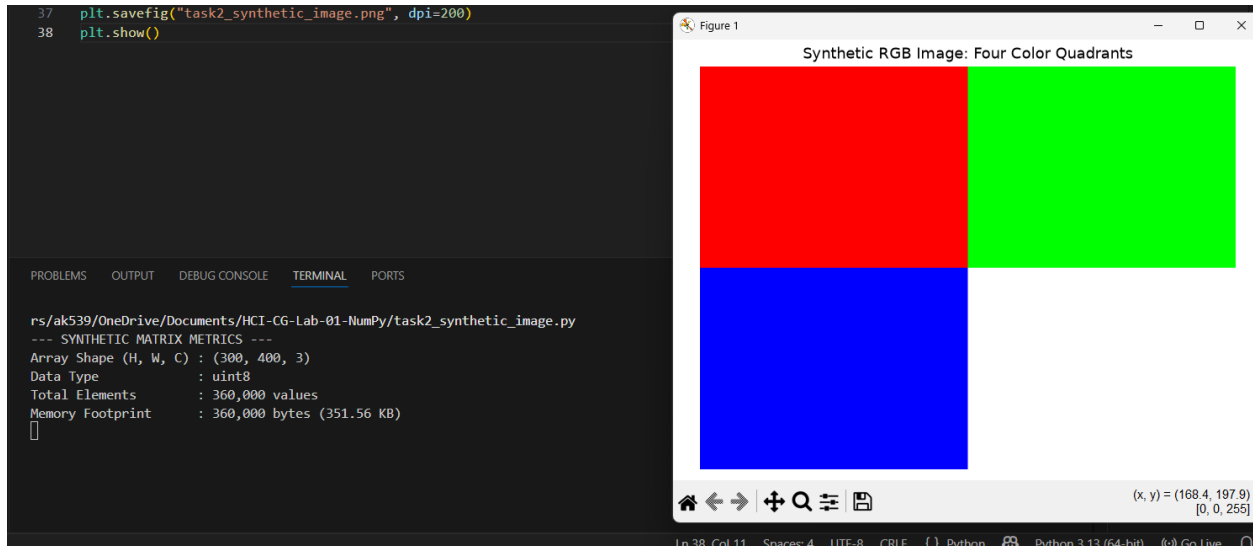
# Bottom-right: White
image[mid_height:, mid_width:] = [255, 255, 255]

print("--- SYNTHETIC MATRIX METRICS ---")
print(f"Array Shape (H, W, C) : {image.shape}")
print(f"Data Type : {image.dtype}")
print(f"Total Elements : {image.size:,} values")
print(f"Memory Footprint : {image.nbytes:,} bytes ({image.nbytes / 1024:.2f} KB)")

plt.figure(figsize=(8, 6))
plt.imshow(image)
plt.title("Synthetic RGB Image: Four Color Quadrants")
plt.axis("off")
plt.tight_layout()
```

```
plt.savefig("task2_synthetic_image.png", dpi=200)
plt.show()
```

- Output screenshot



Task 3: Channel Slicing and Isolation

Objective: To extract red, green, and blue channels from sample.jpg and display their isolated views and grayscale intensity maps.

- Code

```
import os

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

script_folder = os.path.dirname(os.path.abspath(__file__))
image_path = os.path.join(script_folder, "sample.jpg")

if not os.path.exists(image_path):
    print(f"Error: '{image_path}' was not found.")
    print("Place sample.jpg in the same folder as this Python file.")
    raise SystemExit

img = np.array(Image.open(image_path).convert("RGB"))

# Extract 2D color-channel intensity grids
```

```

red_channel = img[:, :, 0]
green_channel = img[:, :, 1]
blue_channel = img[:, :, 2]

# Create RGB arrays containing only one active channel
red_only = np.zeros_like(img)
green_only = np.zeros_like(img)
blue_only = np.zeros_like(img)

red_only[:, :, 0] = red_channel
green_only[:, :, 1] = green_channel
blue_only[:, :, 2] = blue_channel

print("--- CHANNEL EXTRACTION SUMMARY ---")
print(f"Original Image Shape      : {img.shape}")
print(
    f"Red Channel 2D Shape      : {red_channel.shape} | "
    f"Mean Intensity: {red_channel.mean():.2f}"
)
print(
    f"Green Channel 2D Shape     : {green_channel.shape} | "
    f"Mean Intensity: {green_channel.mean():.2f}"
)
print(
    f"Blue Channel 2D Shape      : {blue_channel.shape} | "
    f"Mean Intensity: {blue_channel.mean():.2f}"
)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# Top row: individual color images
axes[0, 0].imshow(red_only)
axes[0, 0].set_title("Red-Only Image")
axes[0, 0].axis("off")

axes[0, 1].imshow(green_only)
axes[0, 1].set_title("Green-Only Image")
axes[0, 1].axis("off")

axes[0, 2].imshow(blue_only)
axes[0, 2].set_title("Blue-Only Image")
axes[0, 2].axis("off")

# Bottom row: grayscale channel intensity maps
axes[1, 0].imshow(red_channel, cmap="gray")

```

```

axes[1, 0].set_title("Red Channel Intensity")
axes[1, 0].axis("off")

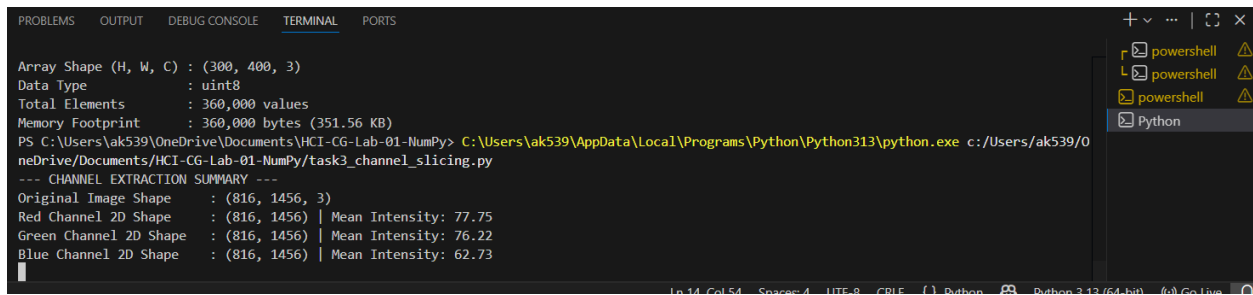
axes[1, 1].imshow(green_channel, cmap="gray")
axes[1, 1].set_title("Green Channel Intensity")
axes[1, 1].axis("off")

axes[1, 2].imshow(blue_channel, cmap="gray")
axes[1, 2].set_title("Blue Channel Intensity")
axes[1, 2].axis("off")

plt.tight_layout()
plt.savefig("task3_channel_slicing.png", dpi=200)
plt.show()

```

-screenshot

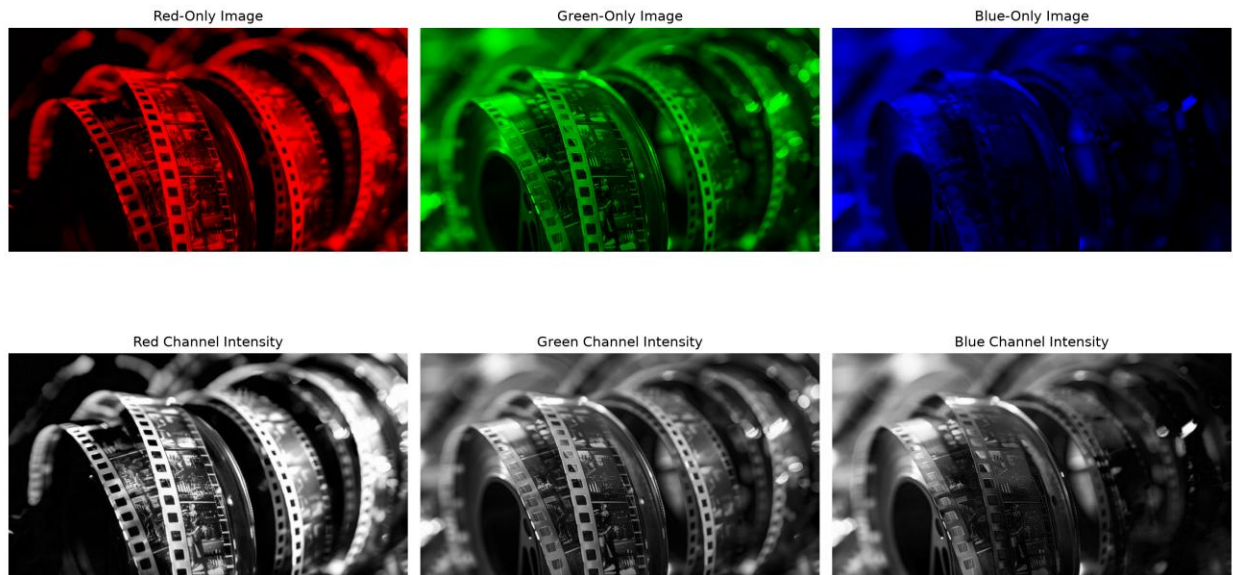


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Array Shape (H, W, C) : (300, 400, 3)
Data Type : uint8
Total Elements : 360,000 values
Memory Footprint : 360,000 bytes (351.56 KB)
PS C:\Users\ak539\OneDrive\Documents\HCI-CG-Lab-01-Numpy> C:\Users\ak539\AppData\Local\Programs\Python\Python313\python.exe c:/Users/ak539/OneDrive/Documents/HCI-CG-Lab-01-Numpy/task3_channel_slicing.py
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (816, 1456, 3)
Red Channel 2D Shape : (816, 1456) | Mean Intensity: 77.75
Green Channel 2D Shape : (816, 1456) | Mean Intensity: 76.22
Blue Channel 2D Shape : (816, 1456) | Mean Intensity: 62.73

```

- 2 × 3 output figure



Task 4: Downsampling and Pixelation

Objective: To reduce an image by taking every 8th pixel using NumPy array slicing and then re-expand it to observe pixelation.

- Code

```
import os

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

image_path = "sample.jpg"
N = 8

if not os.path.exists(image_path):
    print(f"Error: '{image_path}' was not found.")
    print("Place sample.jpg in the same folder as this Python file.")
    raise SystemExit

img = np.array(Image.open(image_path).convert("RGB"))
```

```

original_height, original_width, channels = img.shape

# Take every N-th pixel from rows and columns
downsampled = img[::N, ::N, :]

# Expand pixels to create the blocky/pixelated image
expanded = np.repeat(np.repeat(downsampled, N, axis=0), N, axis=1)

# Crop in case dimensions are not perfectly divisible by N
expanded = expanded[:original_height, :original_width, :]

height_reduction = (1 - downsampled.shape[0] / original_height) * 100
width_reduction = (1 - downsampled.shape[1] / original_width) * 100
memory_savings = (1 - downsampled.nbytes / img.nbytes) * 100

print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")
print(f"Original Shape      : {img.shape} | Memory: {img.nbytes:,} bytes")
print(
    f"Downsampled Shape   : {downsampled.shape} | "
    f"Memory: {downsampled.nbytes:,} bytes"
)
print(f"Re-expanded Shape   : {expanded.shape} | Visual: Blocky Pixelation")
print(
    f"Dimension Reduction: {height_reduction:.2f}% height reduction, "
    f"{width_reduction:.2f}% width reduction"
)
print(f"Memory Savings      : {memory_savings:.2f}% data reduction")

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

axes[0].imshow(img)
axes[0].set_title(f"Original Image\n{img.shape}")
axes[0].axis("off")

axes[1].imshow(downsampled)
axes[1].set_title(f"Downsampled Image (N={N})\n{downsampled.shape}")
axes[1].axis("off")

axes[2].imshow(expanded)
axes[2].set_title(f"Re-expanded / Pixelated\n{expanded.shape}")
axes[2].axis("off")

plt.tight_layout()
plt.savefig("task4_downsampling_pixelation.png", dpi=200)
plt.show()

```

- Output figure



Short Answer: **RGB vs. RGBA Memory Consumption**

RGBA uses 33.33% more memory than RGB because RGB stores three values per pixel: red, green, and blue. RGBA stores an additional fourth value, called the alpha channel, which controls transparency. When uint8 is used, each channel occupies 1 byte. Therefore, an RGB pixel uses 3 bytes, whereas an RGBA pixel uses 4 bytes. The extra 1 byte is one-third of the original 3-byte RGB storage, so RGBA requires 33.33% more memory.